📘 ELECTRONIC SCHOOL SYSTEM

Comprehensive Technical Documentation (C++ Console Application)

1. 📖 Introduction

The **Electronic School System** is a console-based application developed using the **C++ programming language**, designed to simulate the core functionalities of a simple digital learning platform. The system aims to provide users—primarily students or beginner programmers—with an interactive environment where they can study theoretical materials, practice programming exercises, and assess their knowledge through automated testing.

In modern education, digital tools play a crucial role in enhancing learning efficiency and accessibility. While large-scale platforms such as Learning Management Systems (LMS) exist, this project focuses on implementing a simplified version using fundamental programming techniques. The goal is not to replicate complex systems, but rather to demonstrate how core concepts such as control structures, data handling, and modular programming can be used to build a functional educational application.

This project is particularly valuable for beginners in programming, as it combines multiple essential topics into a single cohesive system. It encourages structured thinking, problem-solving, and user interaction design—all within the constraints of a console application.

2. 🎯 Project Objectives

The development of the Electronic School System is guided by several key objectives:

2.1 Reinforcement of C++ Fundamentals

The system serves as a practical application of core C++ concepts, including:

- Variables and data types
- Input and output streams
- Conditional statements
- Loop structures
- Arrays and basic data structures
- Functions and modular programming

By integrating these elements into one project, users gain hands-on experience that strengthens their understanding.

2.2 Development of Structured Programming Skills

A major goal of this project is to promote structured programming practices. The application is divided into logical components (functions), each responsible for a specific task. This modular approach improves readability, maintainability, and scalability.

2.3 Implementation of Testing and Evaluation Logic

The system includes a built-in testing mechanism that evaluates user performance. This involves:

- Question management
- Answer validation
- Score calculation
- Performance evaluation

This objective helps demonstrate how algorithms can be used to assess user input and generate meaningful results.

2.4 Interactive User Experience

Although console-based, the application is designed to be interactive. Users navigate through menus, select options, and receive feedback based on their actions. This interaction enhances engagement and usability.

2.5 Clean and Maintainable Code Design

The project emphasizes:

- Clear function separation
- Meaningful variable names
- Logical program flow
- Avoidance of redundant code

These practices are essential for writing professional-quality software.

3. ⚙ System Overview

The Electronic School System consists of several interconnected modules that work together to provide a complete learning experience. Each module corresponds to a specific functionality within the system.

4. 📚 Learning Materials Module

The Learning Materials module provides users with theoretical knowledge about fundamental programming concepts. This module acts as the educational foundation of the system.

4.1 Topics Covered

Variables and Data Types

Users are introduced to:

- Integer (`int`)
- Floating-point (`float`, `double`)
- Character (`char`)
- Boolean (`bool`)

The system explains how variables store data and how different types affect memory and operations.

Conditional Statements

The system explains decision-making structures:

- `if` statements
- `if-else` constructs
- `switch` statements

These are essential for controlling program flow based on conditions.

Loops

Users learn about repetition structures:

- `for` loops
- `while` loops

The module explains how loops are used to execute code multiple times efficiently.

Arrays

Arrays are introduced as a way to store multiple values in a single variable. Topics include:

- Declaration
- Initialization
- Iteration

4.2 Educational Purpose

This module ensures that users have the necessary theoretical background before attempting exercises or tests. It bridges the gap between theory and practice.

5. 🧩 Exercises Module

The Exercises module provides hands-on practice tasks that reinforce the concepts learned in the Learning Materials section.

5.1 Types of Exercises

Basic Arithmetic Operations

Example:

- Calculate the sum of two numbers

This helps users understand input/output operations and arithmetic expressions.

Conditional Logic Problems

Example:

- Check whether a number is even or odd

This reinforces the use of `if` statements.

Comparative Problems

Example:

- Find the largest of three numbers

This introduces nested conditions and logical reasoning.

5.2 Educational Benefits

The exercises:

- Encourage active learning
- Improve problem-solving skills
- Reinforce theoretical knowledge
- Provide immediate feedback

6. 📝 Automated Testing Module

The Automated Test module is one of the core features of the system. It allows users to assess their understanding through structured quizzes.

6.1 Test Structure

Each test consists of multiple-choice questions with:

- One correct answer

- Three possible options (A, B, C)

6.2 Randomization

Questions are selected randomly to ensure:

- Variety in tests
- Reduced memorization
- Increased fairness

6.3 User Interaction

Users:

- Read each question
- Enter their chosen answer
- Receive immediate validation

6.4 Answer Validation

The system compares user input with the correct answer and:

- Increments the score if correct
- Continues to the next question

6.5 Scoring Mechanism

At the end of the test:

- Total correct answers are calculated
- Results are passed to the statistics module

7. 📊 Grading and Statistics Module

This module evaluates user performance and provides detailed feedback.

7.1 Success Percentage Calculation

The system calculates:

Success Rate (%) = (Correct Answers / Total Questions) × 100

7.2 Grading System

Success Rate	Grade
≥ 90%	Excellent (6)
75% – 89%	Very Good (5)
60% – 74%	Good (4)
50% – 59%	Satisfactory (3)
< 50%	Fail (2)

7.3 Output

The system displays:

- Total correct answers
- Percentage score
- Final grade

7.4 Importance

This module demonstrates:

- Data processing
- Mathematical calculations
- Conditional evaluation

8. 🏗 System Architecture

8.1 Modular Design

The application is divided into functions:

- `showMaterials()`
- `showExercises()`
- `runTest()`
- `showStatistics()`

Each function handles a specific responsibility.

8.2 Data Representation

Questions are stored using a structured format containing:

- Text
- Options
- Correct answer

8.3 Control Flow

The program uses:

- Loops for repetition
- Conditions for decision-making

- Functions for modular execution

9. ▶ Main Function and Program Flow

The `main()` function is the entry point of the application.

9.1 Menu System

Users are presented with options:

1. View Learning Materials
2. Solve Exercises
3. Take Test
4. Exit

9.2 Input Handling

The program reads user input and directs execution accordingly.

9.3 Continuous Execution

The system runs in a loop until the user chooses to exit.

10. 🚀 Installation and Execution

10.1 Requirements

- C++ Compiler (e.g., g++)
- Terminal or Command Prompt

10.2 Compilation

```
g++ main.cpp -o school_system
```

10.3 Execution

```
./school_system
```

11. 👥 Contributors

- NIIvanov24
- FYFilipov24
- VAZARAKYN24
- VHGEORGIEV24

12. 📌 Educational Value

This project is highly beneficial for students because it:

- Combines theory and practice
- Encourages logical thinking
- Demonstrates real-world application of programming

- Builds confidence in coding

13. ⭐ Future Improvements

Potential enhancements include:

- User authentication system
- File-based data storage
- Expanded question bank
- Difficulty levels
- Graphical User Interface (GUI)

14. 📄 Conclusion

The Electronic School System is a comprehensive beginner-level project that effectively demonstrates the application of fundamental C++ programming concepts. Despite being console-based, it provides a structured and interactive environment that simulates real educational systems.

By combining learning materials, exercises, and automated testing into a single platform, the project achieves its goal of delivering an engaging and educational experience. Furthermore, its modular design and scalability make it an excellent foundation for future enhancements and more advanced applications.

15. 📚 References (Optional Section)

- C++ Documentation
- Programming textbooks
- Online coding resources

End of Document